

Du comportement observé sur le site aux produits affichés : comprendre le filtrage collaboratif, le filtrage basé sur le contenu et leur combinaison hybride.

Filtrage collaboratifFiltrage basé sur le contenuFastAPI + Supabase

Guide établi à partir du code actuel du dépôt Me-Antsená.

Il décrit les comportements effectivement implémentés, y compris les replis et les limites.

1. Vue d'ensemble : du site au moteur ML

Le moteur ne choisit pas directement des images sur la page. Il classe des identifiants de produits ; le frontend récupère ensuite les détails de ces produits et dessine les cartes.

1 • **Actions** Consultation, favori, panier, achat

>

2 • **Supabase** Catalogue et table `interactions`

>

3 • **Entraînement** Matrice d'interactions + texte des produits

>

4 • **API ML** Scores collaborative, contenu et hybride

>

5 • **Interface** Hydratation des produits, puis `ProductCard`

Étape 1 — le site enregistre des signaux

Pour un utilisateur connecté, le code enregistre plusieurs types d'actions dans `public.interactions`. Chaque ligne relie un utilisateur à un produit et indique un type d'interaction.

Action sur le site	Type stocké	Poids utilisé à l'entraînement	Où dans le frontend ?
Ouvrir une fiche produit	<code>view</code>	1	<code>pages/ProductDetail.jsx</code>
Ajouter aux favoris	<code>favorite</code>	2	<code>context/FavoritesContext.jsx</code>
Ajouter au panier	<code>add_to_cart</code>	2,5	<code>context/CartContext.jsx</code>
Passer une commande	<code>purchase</code>	5	<code>context/OrdersContext.jsx</code>

Étape 2 — construire les données du modèle

`ml-service/data.py` charge les interactions et construit une matrice **utilisateurs × produits**. Les poids des actions d'un même utilisateur envers un produit sont additionnés. Exemple : $\text{vue} + \text{favori} = 1 + 2 = 3$.

En parallèle, le catalogue fournit les caractéristiques textuelles des produits : nom, catégorie, description et tags. L'entraînement est déclenché par `POST /train` et sauvegarde deux artefacts de modèle. Ce n'est pas un réentraînement à chaque clic.

2. Filtrage collaboratif — « les profils proches »

Cette approche se base sur les comportements partagés entre utilisateurs, pas sur le texte du produit.

Intuition simple

Si plusieurs clients consultent, aiment ou achètent les mêmes produits que vous, le modèle apprend qu'ils ont des goûts qui se ressemblent. Il peut alors proposer des produits appréciés par ces clients, même si vous ne les avez jamais consultés.

Ce que fait le code

1. `data.py` produit la matrice pondérée utilisateur-produit.
2. `CollaborativeFilteringModel.fit()` apprend des vecteurs numériques (« embeddings ») pour chaque utilisateur et chaque produit, par factorisation matricielle. L'algorithme ajuste ces vecteurs sur les interactions observées.
3. Pour un utilisateur, `predict()` calcule le produit scalaire entre son vecteur et les vecteurs de produits. Un score plus élevé signifie que le produit correspond mieux aux préférences apprises.
4. Les produits sont triés par score et les premiers sont retournés à l'API.

```
score collaboratif(u, p) = vecteur_utilisateur(u) · vecteur_produit(p)
```

Exemple illustratif

Vous et d'autres clients achetez des écouteurs et consultez des claviers. Si les clients qui partagent vos achats aiment aussi une souris sans fil, cette souris peut remonter dans vos recommandations, même si sa description ne ressemble pas aux écouteurs.

Avantage : découvre des associations de goûts qui ne sont pas évidentes à partir des seules catégories ou descriptions.

Limite : il faut des interactions et des utilisateurs en nombre suffisant. Un nouvel utilisateur ou un produit jamais vu pendant l'entraînement pose un problème de démarrage à froid.

Démarrage à froid utilisateur prévu dans ce projet

- **Utilisateur déjà connu :** le modèle collaboratif produit ses scores.
- **Utilisateur non connu mais avec des actions récentes :** l'API tente une recommandation basée sur le contenu à partir de ces produits (`source: content_cold_start`).
- **Aucune action connue :** l'API renvoie les produits les plus populaires d'après la matrice d'entraînement (`source: popularity_fallback`).

Ce comportement est dans `ml-service/main.py`. Le modèle collaboratif a aussi un repli popularité intégré, désactivable lors de l'évaluation afin de ne pas gonfler les métriques.

3. Filtrage basé sur le contenu — « des produits similaires »

Cette approche examine les mots et les caractéristiques descriptives des produits. Elle n'a pas besoin de trouver d'autres personnes ayant les mêmes goûts.

Transformer le catalogue en vecteurs

`ContentBasedModel._build_text()` rassemble le nom, la marque, la catégorie, la description et les tags d'un produit en un texte. `TfidfVectorizer` transforme ensuite ce texte en vecteur numérique : les termes informatifs pèsent davantage que les mots fréquents.

Le modèle mesure la similarité cosinus entre les vecteurs. Une valeur plus proche de 1 indique des contenus textuels plus proches.

```
similarité(produit A, produit B) = cosinus(vecteur TF-IDF A, vecteur TF-IDF B)
```

Deux usages dans le service

- `GET /similar/{product_id}` classe les produits les plus proches d'un produit donné.
- `recommend_from_history()` compare les produits candidats aux produits servant de point de départ, puis moyenne leurs similarités. C'est notamment le secours utilisé pour un nouvel utilisateur qui a déjà consulté ou aimé des produits.

Exemple : après avoir consulté un casque audio décrit avec les mots « sans fil », « Bluetooth » et « réduction de bruit », le contenu peut faire remonter d'autres casques partageant ces termes, même si aucun autre client n'a encore acheté ces produits.

Limite actuelle du texte : le prix n'est pas inclus dans la chaîne de caractéristiques. De plus, la récupération actuelle du catalogue initialise la marque à une chaîne vide. Les signaux principaux effectivement disponibles sont donc le nom, la catégorie, la description et les tags.

Un nouveau produit peut être ajouté à l'index existant à la volée avec le vocabulaire déjà appris. Des mots entièrement nouveaux peuvent toutefois être ignorés jusqu'au prochain entraînement.

4. Modèle hybride — combiner les deux signaux

Pour un utilisateur déjà connu, l'API calcule séparément une liste collaborative et une liste basée sur le contenu, puis les combine.

1. Le collaboratif classe des produits selon les habitudes apprises auprès de la communauté.
2. Le contenu cherche des produits proches des produits de la liste collaborative.
3. Le moteur hybride normalise chaque score par le score maximal de sa liste.
4. Il additionne les contributions pondérées, puis trie le résultat.

$$\text{score hybride} = 0,55 \times \text{score collaboratif normalisé} + 0,45 \times \text{score contenu normalisé}$$

La pondération est définie dans `HybridRecommender` : le collaboratif contribue à 55 % et le contenu à 45 %. Les paramètres peuvent être ajustés dans ce modèle.

Transport vers le navigateur

Frontend`getRecommendations()` obtient l'utilisateur connecté

>

`HTTPGET /recommendations/{user_id}?top_k=4`

>

Réponse`!Ds + scores pour collaborative, content et hybrid`

>

SupabaseLe frontend charge nom, prix, image, catégorie et note

>

ReactHome.jsx affiche les produits avec `ProductCard`

Le frontend demande quatre produits par liste sur la page d'accueil. Il hydrate les identifiants renvoyés par le ML avec les fiches Supabase avant d'afficher les cartes.

Si le service ML n'est pas disponible

`src/api/service.js` tente le service ML seulement si l'utilisateur est connecté et si `VITE_ML_SERVICE_URL` est configuré. En cas d'absence ou d'échec, il tente d'abord les recommandations pré-calculées de la table `recommendations`. En dernier recours, il affiche des produits du catalogue. Cette dernière étape est un repli d'affichage, pas une prédiction ML personnalisée.

5. Où les recommandations apparaissent-elles ?

Zone visible	Implémentation constatée	À interpréter comment ?
« Recommandé pour vous »	<code>Home.jsx</code> lit <code>recs.hybrid</code>	Liste hybride, quand les données ML sont disponibles.
« Basé sur vos consultations récentes »	<code>Home.jsx</code> lit <code>recs.content</code>	La donnée provient bien de la liste content de l'API, mais pour un utilisateur connu, celle-ci est calculée à partir des prédictions collaboratives — pas directement de l'historique réel de consultations.
« Produits similaires » sur une fiche	<code>ProductDetail.jsx</code> filtre localement <code>allProducts</code> par égalité de catégorie.	Ce bloc actuel est un repli simple par catégorie et n'appelle pas l'endpoint ML <code>/similar</code> .
« Les clients ont aussi acheté »	La fiche produit alimente cette liste depuis <code>r.collaborative</code> .	Il s'agit des prédictions collaboratives du modèle, pas d'un décompte explicite des achats de produits ensemble.

Important : certains textes de l'interface sont plus précis que le branchement actuel. Pour rendre ces libellés littéralement exacts, il faudrait modifier le code d'affichage ou d'alimentation des listes. Le présent guide distingue donc les noms affichés des algorithmes réellement appelés.

Comment améliorer le système ensuite

- Ajouter des interactions de recherche, de clic sur recommandation et de conversion pour obtenir un signal plus riche.
- Utiliser les interactions réelles récentes comme graines content-based pour les utilisateurs connus, plutôt que les produits déjà prédits par le collaboratif.
- Brancher `getSimilarProducts()` sur la fiche produit si l'on veut que « produits similaires » utilise TF-IDF, et pas uniquement la catégorie.
- Inclure davantage d'attributs fiables (marque, prix, tags normalisés) dans les caractéristiques content-based.
- Mesurer régulièrement Precision@K et Recall@K sur des données de test. Dans le code, seules les interactions fortes (favori, panier, achat) comptent comme pertinentes pour ces mesures.

6. Fichiers à connaître

Collecte et pondération — `src/api/service.js`, `src/pages/ProductDetail.jsx`,
`src/context/CartContext.jsx`, `src/context/FavoritesContext.jsx`, `src/context/OrdersContext.jsx`,
`ml-service/data.py`

Algorithmes — `ml-service/models/collaborative_filtering.py`, `ml-service/models/content_based.py`,
`ml-service/models/hybrid.py`

Entraînement et API — `ml-service/train.py`, `ml-service/main.py`

Affichage du site — `src/api/service.js`, `src/pages/Home.jsx`, `src/pages/ProductDetail.jsx`,
`src/components/ProductCard.jsx`

Structure en base de données — `supabase/schema.sql` (tables `products`, `interactions`, `recommendations`)

Résumé en une phrase

Collaboratif : « des produits appréciés par des personnes qui interagissent comme vous ».

Contenu : « des produits dont le texte ressemble à ceux qui vous intéressent ».

Hybride : « une combinaison pondérée des deux classements ».

Les scores sont des scores de classement internes au modèle, pas des probabilités garanties d'achat. Les résultats dépendent de la fraîcheur des artefacts, de la qualité des données et de la disponibilité du service ML.